

1 Assignment Overview

What you will implement

1. Byte-pair encoding (BPE) tokenizer
2. Transformer language model (LM)
3. The cross-entropy loss function and the AdamW optimizer
4. The training loop, with support for serializing and loading model and optimizer state

DONE

What you will run

1. Train a BPE tokenizer on the TinyStories dataset.
2. Run your trained tokenizer on the dataset to convert it into a sequence of integer IDs.
3. Train a Transformer LM on the TinyStories dataset.
4. Generate samples and evaluate perplexity using the trained Transformer LM.
5. Train models on OpenWebText and submit your attained perplexities to a leaderboard.

DONE

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

In Python, you can use the `ord()` function to convert a single Unicode character into its integer representation. The `chr()` function converts an integer Unicode code point into a string with the corresponding character.

Problem (unicode1): Understanding Unicode (1 point)

- (a) What Unicode character does `chr(0)` return?

Answer: `'\x00'`

- (b) How does this character's string representation `__repr__()` differ from its printed representation?

Answer: The `__repr__()` is designed to be **unambiguous** and is mainly for developers, showing the technical details of the object. On the other hand, the printed representation is meant to be **readable** and user-friendly.

- (c) What happens when this character occurs in text? It may be helpful to play around with the following in your Python interpreter and see if it matches your expectations:

Answer: Code below.

```
>>> chr(0)
'\x00'
>>> print(chr(0))

>>> "this is a test" + chr(0) + "string"
'this is a test\x00string'
>>> print("this is a test" + chr(0) + "string")
this is a teststring
```

2.2 Unicode Encodings

- UTF-8: The Efficient Variable-Length Encoder, It uses 1 to 4 bytes per character.
- UTF-16: The Middle Ground, It uses either 2 or 4 bytes per character.
- UTF-32: The Simple Fixed-Length Encoder, It uses exactly 4 bytes for every character, regardless of what it is.

Problem (unicode2): Unicode Encodings (3 point)

- (a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

Answer: `'\x00'`

- (b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

Example: `b'\xe4\xb8\xad'`.

Answer: This function is incorrect because it attempts to decode each byte individually, which fails for any character encoded as a multi-byte sequence, resulting in a `UnicodeDecodeError`.

- (c) Give a two byte sequence that does not decode to any Unicode character(s).

Example: `b'\xff\xff'`.

Answer: This sequence is invalid because the byte `0xFF` is explicitly forbidden in the UTF-8 standard and does not correspond to any valid lead or continuation byte in the encoding scheme.

2.3 Subword Tokenization

Sennrich et al. [2016] propose to use byte-pair encoding (BPE; Gage, 1994), a compression algorithm that iteratively replaces (“merges”) the most frequent pair of bytes with a single, new unused index.

2.4 BPE Tokenizer Training

Vocabulary initialization

Pre-tokenization Use regular expressions to split the sentence into independent punctuation marks and independent items that may include spaces, and improve the merging effect by counting the occurrence frequency of these independent items.

```
>>> PAT = r"\"(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+\s+(?!S)|\s+\""
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', 'll', ' pre', '-', 'tokenize']
```

Compute BPE merges During the BPE merging process, two **principles** must be followed:

- Do not **cross** pre-tokenization boundaries (this is already guaranteed algorithmically);
- Handle issues with a **reproducible conflict resolution strategy**, such as selecting the lexicographically largest word pair to merge when the occurrence counts are the same.

Special tokens

2.5 Experimenting with BPE Tokenizer Training

Parallelizing pre-tokenization Using the provided segmentation code and the `multiprocessing` library, parallelize the tokenization statistics.

Removing special tokens before pre-tokenization

Optimizing the merging step After performing a statistical pass to obtain a tokenization table with frequencies, perform merging only within that tokenization table.

Low-Resource/Downscaling Tip: Profiling

You should use profiling tools like `cProfile` or `scalene` to identify the bottlenecks in your implementation, and focus on optimizing those.

◇ ◇ ◇

Problem (train_bpe): BPE Tokenizer Training (15 points)

Deliverable: Write a function that, given a path to an input text file, trains a (byte-level) BPE tokenizer. Your BPE training function should handle (at least) the following input parameters:

input_path: `str` Path to a text file with BPE tokenizer training data.

vocab_size: `int` A positive integer that defines the maximum final vocabulary size (including the initial byte vocabulary, vocabulary items produced from merging, and any special tokens).

special_tokens: `list[str]` A list of strings to add to the vocabulary. These special tokens do not otherwise affect BPE training.

Your BPE training function should return the resulting vocabulary and merges:

vocab: `dict[int, bytes]` The tokenizer vocabulary, a mapping from `int` (token ID in the vocabulary) to `bytes` (token bytes).

merges: `list[tuple[bytes, bytes]]` A list of BPE merges produced from training. Each list item is a `tuple` of `bytes` (`<token1>`, `<token2>`), representing that `<token1>` was merged with `<token2>`. The merges should be ordered by order of creation.

◇ ◇ ◇

Insights: BPE Tokenizer Training

Python plus Rust: Python is used for rapid prototyping, while Rust handles the actual deployment of the product.

Glitch Tokens: Due to insufficiently strict data cleaning, some meaningless words—namely **Mojibake**—may appear during the BPE training process. Moreover, because of BPE's **greedy, unsupervised, and non-semantic nature**, a large number of such meaningless word merges

occur, turning them into abnormally long and nonsensical words.

Vocabulary Bloat (Dead Tokens): Because BPE is incremental and never removes merges, the intermediate products of long words permanently occupy the vocabulary, leading to wasted model parameters and tokenization fragmentation. In industry, there are three approaches to address this issue: First, absolutely aggressive data cleaning; second, stricter regex filtering. Both of these are common methods for limiting glitch tokens. The third approach is to switch to the **Unigram** algorithm, which is frequently used by Google-family models (such as T5 and some Llama variants).

Unigram: Unigram works by "subtraction": it first initializes an extremely large vocabulary (e.g., 100,000 words) and **then calculates how much the overall corpus compression rate would be affected if a given word were removed**. Those useless intermediate composite words are directly eliminated and discarded during training.

◇ ◇ ◇

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

- (a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 30 minutes (no GPUs), ≤ 30 GB RAM

Resource used: 10.78 seconds, 2.6GB RAM

Longest token: 'disappointment'

- (b) Profile your code. What part of the tokenizer training process takes the most time?

Answer: `process_chunk_with_special_tokens` takes up the vast majority of the time.

◇ ◇ ◇

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 points)

- (a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

Resource requirements: ≤ 12 hours (no GPUs), ≤ 100 GB RAM

Resource used: 6 minutes, 7.0GB RAM

Longest token: '_____'

- (b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Answer: Because TinyStories has an extremely limited vocabulary, a 10k vocab size forces the BPE tokenizer to over-merge, creating tokens that represent entire multi-word phrases. In contrast, OpenWebText’s diverse internet text requires the BPE algorithm to rely heavily on subwords (like prefixes and suffixes) to represent its massive vocabulary within a 32k limit, resulting in shorter, more fragmented tokens for rare words.

2.6 BPE Tokenizer: Encoding and Decoding

2.6.1 Encoding text

Special tokens.

Memory considerations.

2.6.2 Decoding text

Problem (tokenizer): Implementing the tokenizer (15 points)

Deliverable: Implement a `Tokenizer` class that, given a vocabulary and a list of merges, encodes text into integer IDs and decodes integer IDs into text. Your tokenizer should also support user-provided special tokens (appending them to the vocabulary if they aren’t already there).

Answer: code in appendix.

2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

- (a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer’s compression ratio (bytes/token)?

Answer: The TinyStories tokenizer achieves a higher compression ratio (5.5 bytes/token) because its vocabulary easily memorizes long, frequent whole words from its restricted domain. In contrast, the OpenWebText tokenizer has a lower compression ratio (3.8 bytes/token) as it must rely heavily on shorter subwords to encode a highly diverse vocabulary.

- (b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Answer: Tokenizing OpenWebText with the TinyStories tokenizer causes a severe domain mismatch, forcing the tokenizer to aggressively fragment unfamiliar web text into individual characters or very small byte sequences. Consequently, the token count explodes, resulting in a dramatically lower (worse) compression ratio compared to using the native OpenWebText tokenizer.

- (c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?

Answer: The throughput of the tokenizer depends on the implementation details, but a rough estimate for a modern CPU might be around 100 MB/second. Given the Pile dataset's size of 825GB, it would take approximately 8.25 hours to tokenize the entire dataset.

- (d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype `uint16`. Why is `uint16` an appropriate choice?

Answer: `uint16` is an appropriate choice because it can represent all possible token IDs within the range of 0 to 65,535, which is sufficient for most practical applications and allows for efficient memory usage.

3 Transformer Language Model Architecture

3.1 Transformer LM

3.1.1 Token Embeddings

3.1.2 Pre-norm Transformer Block

3.2 Output Normalization and Embedding

3.3 Remark: Batching, Einsum and Efficient Computation

3.3.1 Mathematical Notation and Memory Ordering

3.4 Basic Building Blocks: Linear and Embedding Modules

3.4.1 Parameter Initialization

3.4.2 Linear Module

Problem (linear): Implementing the linear module (1 point)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation. Your implementation should follow the interface of PyTorch's built-in `nn.Linear` module, except for not having a `bias` argument or parameter. We recommend the following interface:

`def __init__(self, in_features, out_features, device=None, dtype=None)` Construct a linear transformation module. This function should accept the following parameters:

`in_features: int` final dimension of the input

`out_features: int` final dimension of the output

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

`def forward(self, x: torch.Tensor) -> torch.Tensor` Apply the linear transformation to the input.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- construct and store your parameter as W (not W^T) for memory ordering reasons, putting it in an `nn.Parameter`
- of course, don't use `nn.Linear` or `nn.functional.linear`

Code below;

3.4.3 Embedding Module

Problem (embedding): Implement the embedding module (1 point)

Deliverable: Implement the `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. Your implementation should follow the interface of PyTorch's built-in `nn.Embedding` module. We recommend the following interface:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

Construct an embedding module. This function should accept the following parameters:

`num_embeddings: int` Size of the vocabulary

`embedding_dim: int` Dimension of the embedding vectors, i.e., d_{model}

`device: torch.device | None = None` Device to store the parameters on

`dtype: torch.dtype | None = None` Data type of the parameters

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

Lookup the embedding vectors for the given token IDs.

Make sure to:

- subclass `nn.Module`
- call the superclass constructor
- initialize your embedding matrix as a `nn.Parameter`
- store the embedding matrix with the `d_model` being the final dimension
- of course, don't use `nn.Embedding` or `nn.functional.embedding`

Again, use the settings from above for initialization, and use `torch.nn.init.trunc_normal_` to initialize the weights.

To test your implementation, implement the test adapter at `[adapters.run_embedding]`. Then, run `uv run pytest -k test_embedding`.

3.5 Pre-Norm Transformer Block

3.5.1 Root Mean Square Layer Normalization

Problem (rmsnorm): Root Mean Square Layer Normalization (1 point)

Deliverable: Implement `RMSNorm` as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

Construct the `RMSNorm` module. This function should accept the following parameters:

`d_model: int` Hidden dimension of the model

```

eps: float = 1e-5 Epsilon value for numerical stability

device: torch.device | None = None Device to store the parameters on

dtype: torch.dtype | None = None Data type of the parameters

```

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(batch_size, sequence_length, d_model)` and return a tensor of the same shape.

Note: Remember to upcast your input to `torch.float32` before performing the normalization (and later downcast to the original dtype), as described above.

To test your implementation, implement the test adapter at `[adapters.run_rmsnorm]`. Then, run `uv run pytest -k test_rmsnorm`.

3.5.2 Position-Wise Feed-Forward Network

Problem (positionwise_feedforward): Implement the position-wise feed-forward network (2 points)

Deliverable: Implement the SwiGLU feed-forward network, composed of a SiLU activation function and a GLU.

Note: in this particular case, you should feel free to use `torch.sigmoid` in your implementation for numerical stability.

You should set d_{ff} to approximately $\frac{8}{3} \times d_{model}$ in your implementation, while ensuring that the dimensionality of the inner feed-forward layer is a multiple of 64 to make good use of your hardware. To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_swiglu]`. Then, run `uv run pytest -k test_swiglu` to test your implementation.

3.5.3 Relative Positional Embeddings

Problem (rope): Implement RoPE (2 points)

Deliverable: Implement a class `RotaryPositionalEmbedding` that applies RoPE to the input tensor. The following interface is recommended:

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

Construct the RoPE module and create buffers if needed.

```

theta: float  $\Theta$  value for the RoPE

d_k: int dimension of query and key vectors

max_seq_len: int Maximum sequence length that will be inputted

device: torch.device | None = None Device to store the buffer on

```

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

Process an input tensor of shape `(..., seq_len, d_k)` and return a tensor of the same shape. Note that you should tolerate `x` with an arbitrary number of batch dimensions. You should assume that the token positions are a tensor of shape `(..., seq_len)` specifying the token positions of `x` along the sequence dimension.

You should use the token positions to slice your (possibly precomputed) `cos` and `sin` tensors along the sequence dimension.

To test your implementation, complete `[adapters.run_rope]` and make sure it passes `uv run pytest -k test_rope`.

3.5.4 Scaled Dot-Product Attention

Problem (softmax): Implement softmax (1 point)

Deliverable: Write a function to apply the softmax operation on a tensor. Your function should take two parameters: a tensor and a *dimension* `i`, and apply softmax to the `i`-th dimension of the input tensor. The output tensor should have the same shape as the input tensor, but its `i`-th dimension will now have a normalized probability distribution. Use the trick of subtracting the maximum value in the `i`-th dimension from all elements of the `i`-th dimension to avoid numerical stability issues.

To test your implementation, complete `[adapters.run_softmax]` and make sure it passes `uv run pytest -k test_softmax_matches_pytorch`.

◇ ◇ ◇

Problem (scaled_dot_product_attention): Implement scaled dot-product attention (5 points)

Deliverable: Implement the scaled dot-product attention function. Your implementation should handle keys and queries of shape `(batch_size, ..., seq_len, d_k)` and values of shape `(batch_size, ..., seq_len, d_v)`, where `...` represents any number of other batch-like dimensions (if provided). The implementation should return an output with the shape `(batch_size, ..., d_v)`. See section 3.3 for a discussion on batch-like dimensions.

Your implementation should also support an optional user-provided boolean mask of shape `(seq_len, seq_len)`. The attention probabilities of positions with a mask value of `True` should collectively sum to 1, and the attention probabilities of positions with a mask value of `False` should be zero.

To test your implementation against our provided tests, you will need to implement the test adapter at `[adapters.run_scaled_dot_product_attention]`.

`uv run pytest -k test_scaled_dot_product_attention` tests your implementation on third-order input tensors, while `uv run pytest -k test_4d_scaled_dot_product_attention`

tests your implementation on fourth-order input tensors.

3.5.5 Causal Multi-Head Self-Attention

Problem (multihead_self_attention):

Implement causal multi-head self-attention (5 points)

Deliverable: Implement causal multi-head self-attention as a `torch.nn.Module`. Your implementation should accept (at least) the following parameters:

`d_model`: `int` Dimensionality of the Transformer block inputs.

`num_heads`: `int` Number of heads to use in multi-head self-attention.

Following Vaswani et al. [2017], set $d_k = d_v = d_{\text{model}}/h$.

To test your implementation against our provided tests, implement the test adapter at `[adapters.run_multihead_self_attention]`. Then, run `uv run pytest -k test_multihead_self_attention` to test your implementation.

3.6 The Full Transformer LM

Problem (transformer_block): Implement the Transformer block (3 points)

Implement the pre-norm Transformer block as described in §3.5 and illustrated in Figure 2. Your Transformer block should accept (at least) the following parameters.

`d_model`: `int` Dimensionality of the Transformer block inputs.

`num_heads`: `int` Number of heads to use in multi-head self-attention.

`d_ff`: `int` Dimensionality of the position-wise feed-forward inner layer.

To test your implementation, implement the adapter `[adapters.run_transformer_block]`. Then run `uv run pytest -k test_transformer_block` to test your implementation.

Deliverable: Transformer block code that passes the provided tests.

◇ ◇ ◇

Problem (transformer_lm): Implementing the Transformer LM (3 points)

Time to put it all together! Implement the Transformer language model as described in §3.1 and illustrated in Figure 1. At minimum, your implementation should accept all the aforementioned construction parameters for the Transformer block, as well as these additional parameters:

`vocab_size`: `int` The size of the vocabulary, necessary for determining the dimensionality of the token embedding matrix.

`context_length`: `int` The maximum context length, necessary for determining the dimen-

sionality of the position embedding matrix.

`num_layers`: `int` The number of Transformer blocks to use.

To test your implementation against our provided tests, you will first need to implement the test adapter at `[adapters.run_transformer_lm]`. Then, run `uv run pytest -k test_transformer_lm` to test your implementation.

Deliverable: A Transformer LM module that passes the above tests.

◇ ◇ ◇

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

Subproblem(a)

Consider GPT-2 XL, which has the following configuration:

```
vocab_size      50,257
context_length  1,024
num_layers      48
d_model         1,600
num_heads       25
d_ff            6,400
```

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

Answer:

A. Token Embeddings

- **Calculation:** $\text{vocab_size} \times d_{\text{model}} = 50,257 \times 1,600 = \mathbf{80,411,200}$
- **Function:** Maps discrete word IDs to continuous numerical vectors; this is the first step in the model's understanding of language.

B. Transformer Blocks (48 layers in total)

Each Block contains:

1. Attention Projections:

- Four matrices W_q, W_k, W_v, W_o , each of size $d_{\text{model}} \times d_{\text{model}}$.
- **Parameters:** $4 \times (1,600 \times 1,600) = 10,240,000$

2. SwiGLU Feed-Forward Network (FFN):

- According to your implementation, it contains W_1, W_3 (up-projection) and W_2 (down-projection).
- **Parameters:** $3 \times (d_{\text{model}} \times d_{\text{ff}}) = 3 \times (1,600 \times 6,400) = 30,720,000$

3. Layer Normalization (RMSNorm):

- **ln1 and ln2** each have a learnable scale parameter of length d_{model} .
- **Parameters:** $2 \times 1,600 = 3,200$

Subtotal per layer: $10,240,000 + 30,720,000 + 3,200 = 40,963,200$

Total for 48 layers: $48 \times 40,963,200 = \mathbf{1,966,233,600}$

C. Final Output Layer (Final Norm & LM Head)

- **Final RMSNorm:** 1,600
- **LM Head:** Maps the hidden state back to the vocabulary, $d_{\text{model}} \times \text{vocab_size} = 1,600 \times 50,257 = 80,411,200$
- **Function:** Converts the high-dimensional features learned by the model back into a probability distribution over words, used to predict the next word.

Total Parameters:

$80,411,200 + 1,966,233,600 + 80,411,200 = \mathbf{2,127,056,000}$

Memory Cost(FP32):

$2,127,056,000 \times 4 \text{ Bytes} \approx \mathbf{8,508,230,400 \text{ Bytes}}$

Subproblem(b)

Identify the matrix multiplies required to complete a forward pass of our GPT-2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has `context_length` tokens.

Answer:

A. The Foyer (Entrance)

- **Token Embedding:** Pure memory lookup to fetch the 1,600-dimensional vector. No arithmetic operations. **FLOPs: 0.**
- **Positional Encoding (RoPE):** Multiplies features by sin and cos. Approx 4 operations per element.
FLOPs: $4 \times s \times d = 4 \times 1024 \times 1600 \approx \mathbf{6.55 \text{ MFLOPs.}}$

B. Transformer Blocks (Single Layer)

Each Block contains:

1. Attention Mechanism:

- **RMSNorm 1:** Approx 5 ops per element. **FLOPs:** $5 \times s \times d \approx 8 \text{ MFLOPs.}$
- **Q, K, V Projections (W_q, W_k, W_v):** Complexity $\mathcal{O}(sd^2)$.
FLOPs: $3 \times 2sd^2 \approx \mathbf{15.73 \text{ GFLOPs.}}$
- **Attention Scores & Value Extraction:** $Q \times K^\top$ and $\text{Score} \times V$.
FLOPs: $2s^2d + 2s^2d \approx \mathbf{6.71 \text{ GFLOPs.}}$

- **Internal Softmax:** Approx 3 ops per element. **FLOPs:** $3 \times s^2 \times h \approx 78.64$ MFLOPs.
- **Output Projection (W_o):** **FLOPs:** $2sd^2 \approx 5.24$ GFLOPs.
- **Residual Add 1:** **FLOPs:** $s \times d \approx 1.63$ MFLOPs.

2. SwiGLU Feed-Forward Network (FFN):

- **RMSNorm 2:** **FLOPs:** ≈ 8 MFLOPs.
- **Up/Gate (W_1, W_3) & Down (W_2) Projections:**
FLOPs: $(2 \times 2sdd_{\text{ff}}) + (2sdd_{\text{ff}}) = 6sdd_{\text{ff}} \approx 62.91$ GFLOPs.
- **SiLU & Gating:** Approx 5 ops per element. **FLOPs:** $5 \times s \times d_{\text{ff}} \approx 32.76$ MFLOPs.
- **Residual Add 2:** **FLOPs:** ≈ 1.63 MFLOPs.

Single Block Summary:

Large Matrix Multiplications: ≈ 90.60 GFLOPs

Minor Operations (Norms/Acts/Adds): ≈ 137.21 MFLOPs (Only $\approx 0.15\%$ of a single layer)

C. Scaling Up (48 Layers)

- **Core Matrix Operations:** 48×90.60 GFLOPs $\approx 4,348.65$ GFLOPs.
- **Minor Operations:** 48×137.21 MFLOPs ≈ 6.58 GFLOPs.

D. The Exit (Final Output)

- **Final RMSNorm:** **FLOPs:** ≈ 8 MFLOPs.
- **LM Head Projection:** Maps 1600D features to vocabulary space.
FLOPs: $2 \times s \times d \times V = 2 \times 1024 \times 1600 \times 50,257 \approx 164.68$ GFLOPs.
- **Final Softmax:** Approx 3 ops per element.
FLOPs: $3 \times 1024 \times 50,257 \approx 154.38$ MFLOPs.

Total FLOPs (Forward Pass):

$4348.65 + 6.58 + 164.68 + 0.15 \approx 4,520$ GFLOPs (approx. **4.52** TFLOPs)

Subproblem(c)

Based on your analysis above, which parts of the model require the most FLOPs?

Answer:

Based on the FLOPs analysis, the **Feed-Forward Network (FFN)** requires the absolute majority of the computation during a standard forward pass, rather than the Attention mechanism.

Key Insights:

1. The True Computational Bottleneck is the FFN, not Attention:

Due to the massive matrix dimension expansion (typically $d \rightarrow 4d$, or scaling to d_{ff} in SwiGLU), the computational cost of the FFN is nearly 3 times that of the attention linear

projections (Q, K, V, O). This is precisely why cutting-edge large language models heavily research **Mixture of Experts (MoE)** architectures—sparsifying the FFN activations instantly saves a massive amount of compute!

2. The $\mathcal{O}(s^2)$ Complexity is Only Dominant at Long Contexts:

At a standard sequence length of $s = 1024$, the quadratic attention operations (like $QK^\top$) actually account for a negligible fraction of the total computation. It is only when the sequence length s scales up to tens or hundreds of thousands of tokens that this $\mathcal{O}(s^2)$ bottleneck overtakes the FFN to become the primary computational constraint.

Subproblem(d)

Repeat your analysis with GPT-2 small (12 layers, 768 `d_model`, 12 heads), GPT-2 medium (24 layers, 1024 `d_model`, 16 heads), and GPT-2 large (36 layers, 1280 `d_model`, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

Deliverable: For each model, provide a breakdown of model components and its associated FLOPs (as a proportion of the total FLOPs required for a forward pass). In addition, provide a one-to-two sentence description of how varying the model size changes the proportional FLOPs of each component.

Answer:

Assuming the standard sequence length $s = 1024$, vocabulary size $V = 50,257$, and $d_{\text{ff}} = 4 \times d_{\text{model}}$, we calculate the core FLOPs (ignoring minor operations like Norms/Residuals which account for $< 0.2\%$ of compute).

1. GPT-2 Small ($L = 12, d_{\text{model}} = 768$)

- **Total FLOPs:** ≈ 349.6 GFLOPs
- **Attention (Proj + Scores):** ≈ 96.7 GFLOPs (**27.7%**)
- **FFN (SwiGLU):** ≈ 173.9 GFLOPs (**49.7%**)
- **LM Head:** ≈ 79.0 GFLOPs (**22.6%**)

2. GPT-2 Medium ($L = 24, d_{\text{model}} = 1024$)

- **Total FLOPs:** $\approx 1,033.1$ GFLOPs
- **Attention (Proj + Scores):** ≈ 309.2 GFLOPs (**29.9%**)
- **FFN (SwiGLU):** ≈ 618.5 GFLOPs (**59.9%**)
- **LM Head:** ≈ 105.4 GFLOPs (**10.2%**)

3. GPT-2 Large ($L = 36, d_{\text{model}} = 1280$)

- **Total FLOPs:** $\approx 2,257.8$ GFLOPs
- **Attention (Proj + Scores):** ≈ 676.5 GFLOPs (**30.0%**)

- **FFN (SwiGLU):** $\approx 1,449.6$ GFLOPs (**64.2%**)
- **LM Head:** ≈ 131.7 GFLOPs (**5.8%**)

Trend Description:

As the model size increases, the computational cost is heavily dominated by the internal Transformer blocks (specifically the FFN and Attention projections, which scale quadratically with the hidden dimension, $\mathcal{O}(L \cdot s \cdot d^2)$). Consequently, the LM Head (which scales only linearly with the hidden dimension, $\mathcal{O}(s \cdot d \cdot V)$) takes up a proportionally smaller fraction of the total FLOPs, plummeting from nearly 23% in the Small model to roughly 6% in the Large model.

Subproblem(e)

Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

Answer:

Increasing the context length s from 1,024 to 16,384 (a $16\times$ increase) causes the total FLOPs to skyrocket from roughly 4.5 TFLOPs to ≈ 149.5 **TFLOPs** (an approximate $33\times$ increase). The relative contributions of the model components change fundamentally: the quadratic $\mathcal{O}(s^2d)$ attention score computation overtakes the FFN to become the absolute computational bottleneck.

Detailed Breakdown:

- **Linear Scaling Components ($\mathcal{O}(s)$, scaled by $16\times$):**
 - **FFN:** ≈ 1006.6 GFLOPs per layer $\times 48 \approx$ **48.3 TFLOPs (32.3%)**
 - **Linear Projections (Q, K, V, O):** ≈ 335.5 GFLOPs per layer $\times 48 \approx$ **16.1 TFLOPs (10.8%)**
 - **LM Head:** $\approx$ **2.6 TFLOPs (1.8%)**
- **Quadratic Scaling Component ($\mathcal{O}(s^2)$, scaled by $256\times$):**
 - **Attention Scores ($QK^\top$ & SV):** The original 6.71 GFLOPs per layer explodes to $\approx 1,718$ GFLOPs per layer!
Total across 48 layers: $\approx$ **82.5 TFLOPs (55.1%)**

Conclusion: At $s = 16,384$, the attention mechanism alone accounts for over **65%** of the total compute (Projections + Scores), completely dwarfing the FFN. This demonstrates why techniques like FlashAttention, Ring Attention, or Sparse Attention are strictly required for long-context language models.